

GCC 128 — INTELIGÊNCIA ARTIFICIAL

Redes Neurais com MLPClassifier

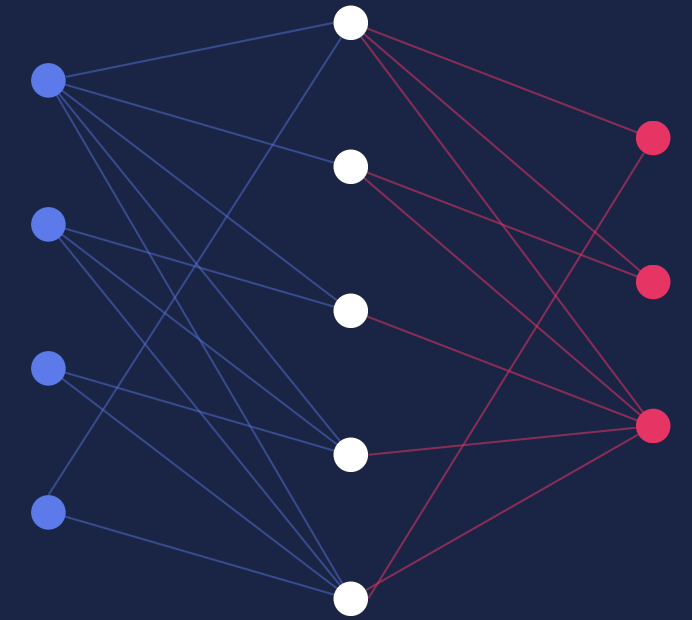
Comparando o MLPClassifier do Scikit-learn com o KNN do TP01, nas bases Iris e Wine.

PROFESSOR

Ahmed Ali Abdalla Esmin

ALUNO

Tobias Maugus Bueno Cougo



Objetivo da atividade

Aplicar o MLPClassifier a duas bases e comparar seu desempenho com o KNN do TP01.



Classificar com MLP

Aplicar o MLPClassifier do Scikit-learn às bases Iris e Wine.



Evitar vazamento de dados

Padronizar com StandardScaler dentro de um Pipeline, ajustado só no treino.



Avaliar com métricas

Gerar matriz de confusão e métricas de precisão, revocação e acurácia.



Comparar com o KNN (TP01)

Usar a mesma divisão e semente no Iris para uma comparação direta e justa.

Como um MLP classifica

1

Camada de entrada

Recebe os atributos numéricos da amostra (ex.: medidas da flor ou do vinho).

2

Camada oculta

Combina as entradas com pesos e aplica uma função de ativação (ReLU), aprendendo padrões não lineares.

3

Camada de saída

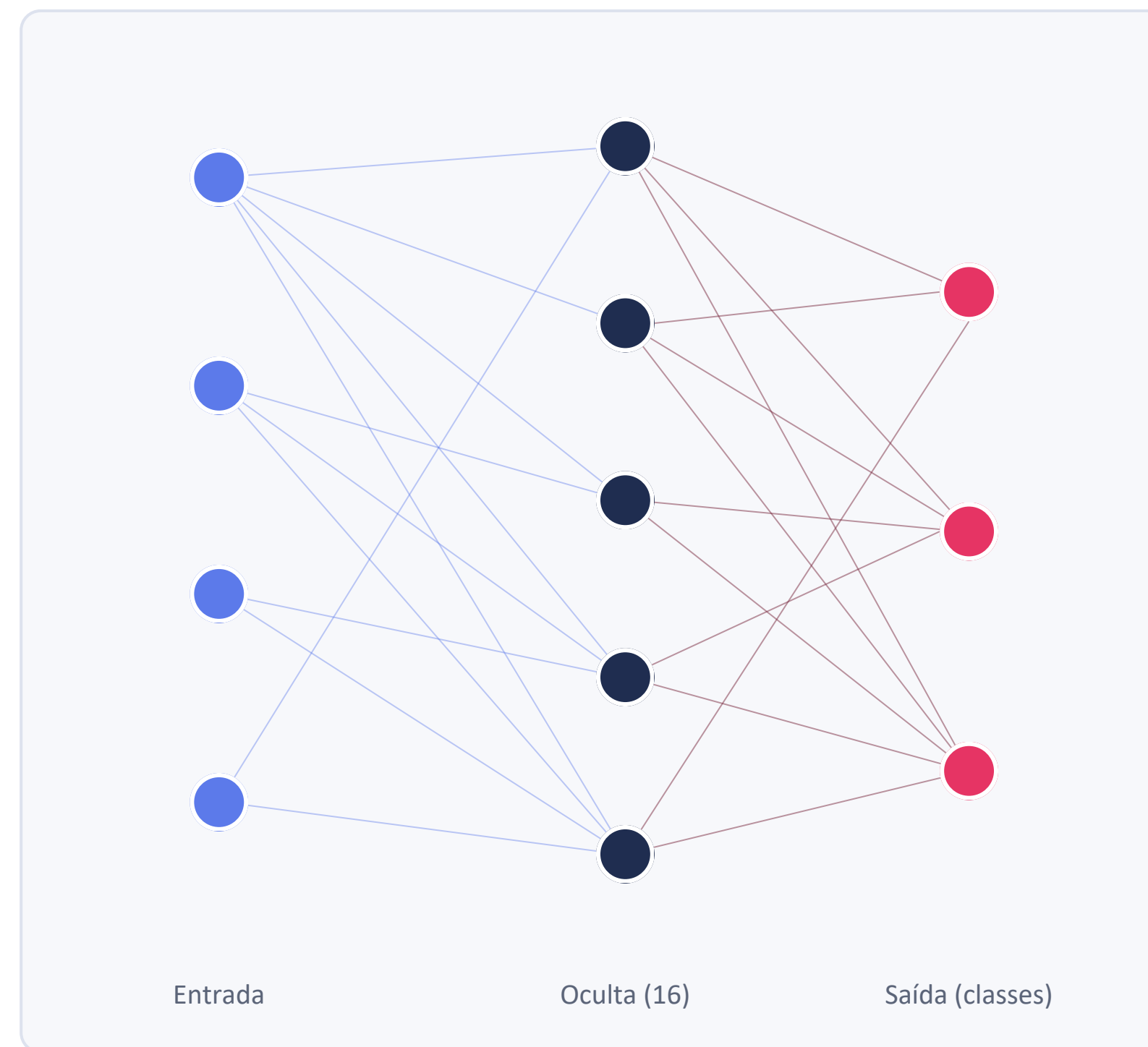
Produz uma pontuação para cada classe possível — a maior indica a previsão.

4

Treinamento

Um otimizador ajusta os pesos da rede para reduzir o erro nas previsões (aqui, o L-BFGS).

Diferente do KNN, o MLP aprende pesos durante o treino, em vez de guardar os dados.



Protocolo experimental

Mesmo pipeline aplicado às duas bases, com a mesma divisão e semente do TP01 no Iris.

1**Divisão dos dados**

70% treino / 30% teste, estratificada por classe, com `random_state=42` — igual ao TP01 no Iris.

**2****StandardScaler**

Ajustado apenas no treino, dentro do Pipeline, evitando vazamento de informação do teste.

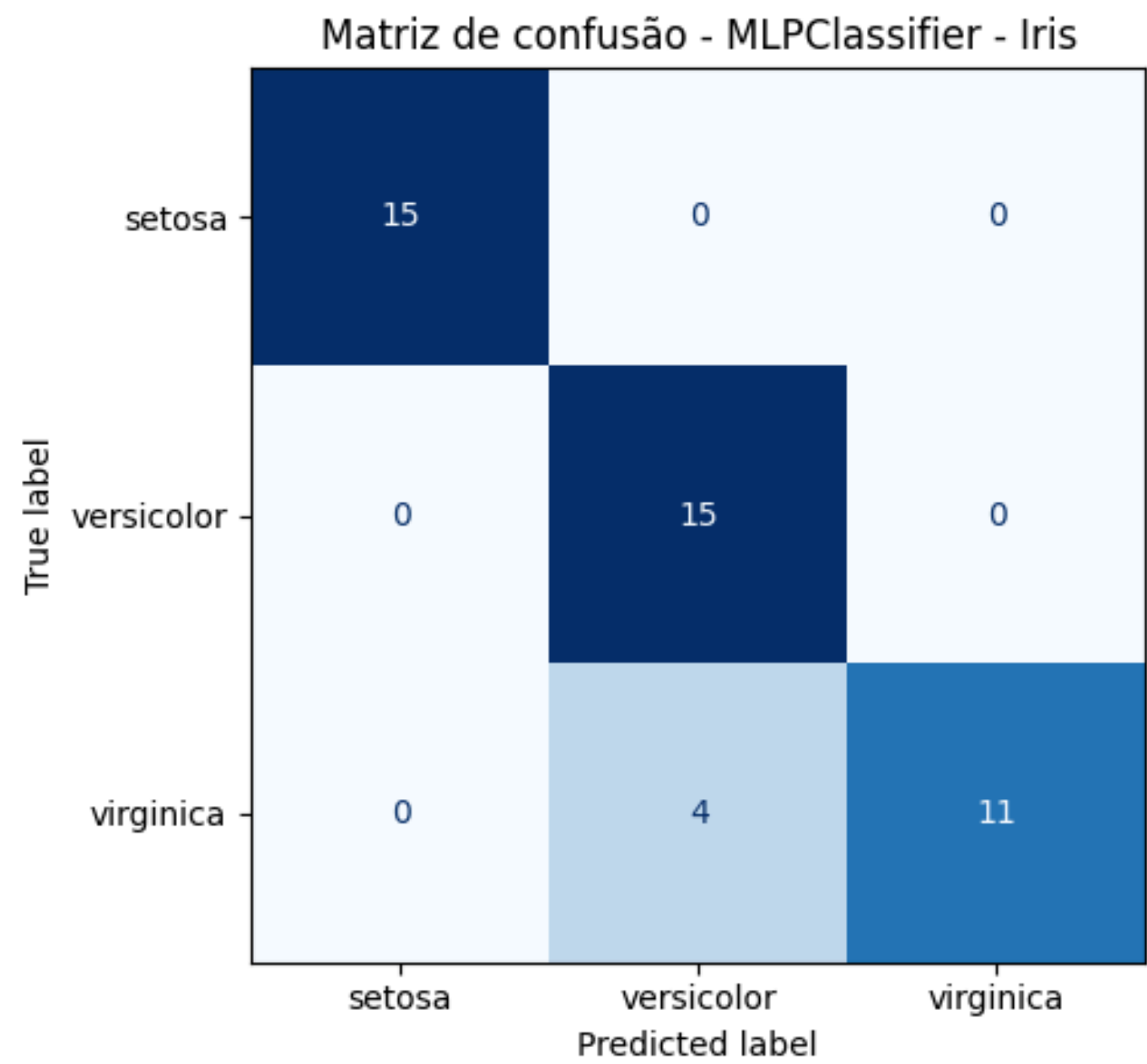
**3****MLPClassifier**

1 camada oculta (16 neurônios), ativação ReLU, solver L-BFGS, `max_iter=2000`.

```
modelo = Pipeline([
    ("escala", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(16,), activation="relu",
        solver="lbfgs", alpha=1e-4, max_iter=2000,
        random_state=42,
    )),
])
```

Resultados na base Iris

Único erro sistemático: 4 das 15 amostras de virginica confundidas com versicolor.



0,9111

Acurácia

0,9298

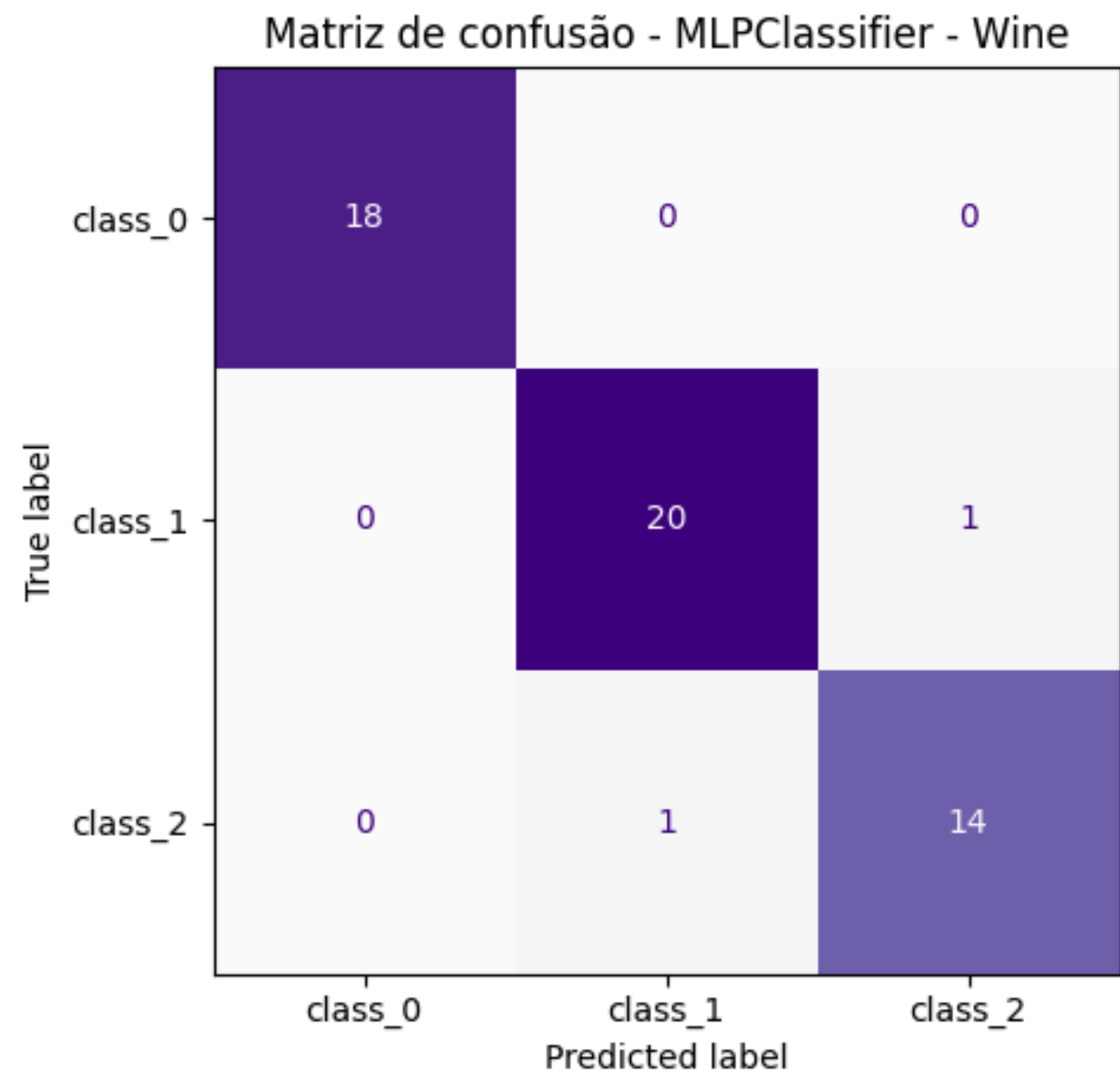
Precisão macro

0,9111

Revocação macro

Resultados na base Wine

Apenas 1 troca em cada sentido entre class_1 e class_2; class_0 100% correta.



0,9630

Acurácia

0,9619

Precisão macro

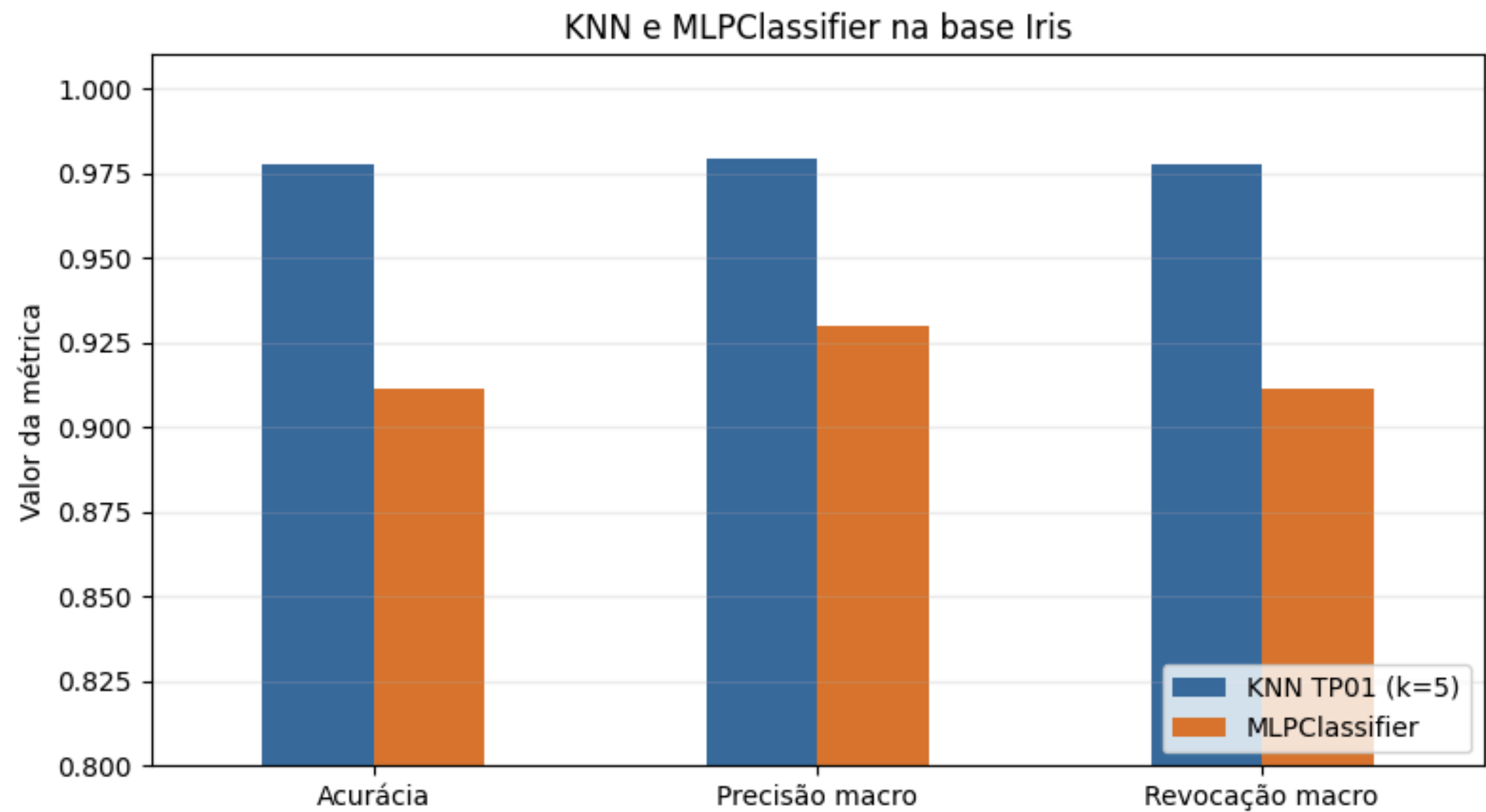
0,9619

Revocação macro

COMPARAÇÃO

MLP vs. KNN (TP01) no Iris

Mesma divisão e semente (random_state=42) nas duas execuções, para uma comparação justa.



RESULTADO

KNN venceu

- 0,0667 Acurácia
- 0,0494 Precisão macro
- 0,0667 Revocação macro

Diferença de poucas amostras (45 no teste) — não implica superioridade geral do algoritmo.

Por que o KNN levou vantagem aqui?

A base Iris é pequena e bem separada — e o MLP usado não teve ajuste de hiperparâmetros.

KNN

- Sem fase de treino real — é um algoritmo “lazy”
- Custo de predição cresce com o tamanho da base
- Não exige padronização obrigatória
- Sem arquitetura para ajustar — poucos hiperparâmetros

MLP

- Aprende pesos durante o treino (mais custoso)
- Predição rápida depois, via operações matriciais
- Exige padronização dos atributos
- Precisa escolher arquitetura, ativação e solver

CONCLUSÃO

O que aprendemos



Nenhum algoritmo é universalmente melhor

No Iris, o KNN levou vantagem; no Wine, o MLP teve ótimo desempenho (96,3%) sem baseline de comparação.



O cenário de uso importa mais que a métrica isolada

Tamanho da base, custo de treino/predição, necessidade de padronização e de ajuste de hiperparâmetros pesam na escolha.



Métricas por classe revelam mais que a acurácia

As matrizes de confusão mostraram exatamente onde cada modelo erra — essencial para interpretar os resultados.

Obrigado!

Tobias Maugus Bueno Cougo